

UNIVERSITATEA POLITEHNICĂ DIN BUCUREȘTI
FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
DEPARTAMENTUL DE CALCULATOARE



PROIECT DE DISERTAȚIE

Scheduling de Task-uri În Sisteme Distribuite

Bogdan Dumitrescu

Coordonator științific:

Prof. dr. ing. Radu-Ioan Ciobanu

BUCUREȘTI

2026

SINOPSIS

Această lucrare abordează problema planificării (*scheduling*) task-urilor în sisteme distribuite, cu accent pe execuția de *workflows* compuse din mai multe componente dependente. În practică, multe aplicații moderne sunt formate din pași care depind unii de alții (de exemplu, etape de procesare, microservicii sau job-uri), iar performanța totală este influențată atât de ordinea de execuție impusă de dependențe, cât și de decizia de plasare pe noduri (*node placement*) într-un cluster heterogen. Pornind de la idei clasice din literatura de planificare (de exemplu, planificare centralizată, planificare distribuită, politici de echitate și mecanisme bazate pe constrângeri), lucrarea propune un model de execuție în care workflow-urile sunt reprezentate ca grafuri aciclice orientate (DAG), iar nodurile clusterului au capacități diferite (CPU-optimized, IO-optimized, GPU etc.).

Contribuția principală este definirea unei abordări de planificare orientată pe workflow pentru Kubernetes: etichetarea capacităților nodurilor, definirea compatibilităților ca *hard constraints*, introducerea unei ordini de preferințe (best/next-best) pentru plasare sub contendență și explorarea unui mecanism de *warm-instance caching* (hot/cold) pentru reducerea penalităților de *cold start*. Rezultatul urmărit este reducerea latenței de start și îmbunătățirea performanței end-to-end a workflow-urilor în clustere heterogene.

CUPRINS

1	Introducere	3
1.1	Context	3
1.2	Problema	3
1.3	Motivație: de ce nu e suficient Argo + kube-scheduler default	4
1.4	Obiective	4
1.5	Întrebări de cercetare și ipoteze	5
1.6	Soluția propusă	5
1.7	Structura lucrării	6
2	State of the Art	7
2.1	Planificare orientată pe workflow (DAG) pe resurse heterogene	7
2.2	Interfața workflow-to-scheduler: transmiterea contextului către scheduler . .	7
2.3	Co-scheduling (gang) pentru stadii fork-join	7
2.4	Admission control și queueing la nivel de workflow	8
2.5	Localitatea datelor vs. localitatea imaginilor	8
2.6	Cold start și distribuția imaginilor: impact asupra latenței de pornire	8
3	Arhitectură și implementare	9
3.1	Viziune generală	9
3.2	Model: workflow, task și semnale	9
3.3	Interfața workflow-to-scheduler (CRD/Annotations)	10
3.4	Politica workflow-aware: critical-path / HEFT-inspired scoring	10
3.5	Cold-start aware scheduling	10
3.6	Stage-level co-scheduling (gang) pentru fork-join	11
3.7	Learning loop: explore/exploit pentru workflows repetate	11

3.8	Admission control	12
3.9	Optimizare globală sub concurență: priorități și (ulterior) preemptie	12
4	Evaluare experimentală	13
4.1	Setup experimental	13
4.2	Variante comparate	13
4.3	Metrici	13
4.4	Ipoteze testate	14
4.5	Amenințări la validitate	14
5	Limitări și direcții ulterioare	15

1 INTRODUCERE

1.1 Context

Planificarea task-urilor în sisteme distribuite reprezintă procesul prin care sistemul decide **când** și **unde** vor rula unități de execuție într-un cluster, sub constrângeri de resurse și politici precum latența, throughput-ul, utilizarea resurselor și echitatea. În practică, aceste decizii influențează direct performanța aplicațiilor și costurile operaționale. Mai mult, clusterelor moderne sunt frecvent **heterogene**, incluzând noduri cu profile diferite (CPU-optimized, IO-optimized, GPU, high-memory), ceea ce face ca aceeași componentă să poată avea timpi de execuție și costuri de pornire semnificativ diferite în funcție de plasare.

În plus, multe aplicații nu constau dintr-un singur job independent, ci din **workflows** compuse din mai multe etape cu dependențe, care pot fi modelate ca grafuri aciclice orientate (DAG). În Kubernetes, orchestratoare precum Argo Workflows[2, 1] gestionează în principal *dependency scheduling* (determină ce task devine eligibil în funcție de dependențe) în timp ce decizia efectivă de plasare a fiecărui pod (*node placement*) este realizată de kube-scheduler[6, 8], în mod local, fără vizibilitate asupra structurii DAG-ului sau a obiectivului end-to-end al workflow-ului.

Această separare de responsabilități creează o oportunitate de cercetare și inginerie: introducerea unui strat de **workflow-aware scheduling** care aduce informația de DAG (de exemplu, *critical path*) și penalitățile de pornire (*cold start*) în decizia de plasare, cu scopul de a reduce latența end-to-end și variabilitatea execuției pentru workflows rulate pe cluster Kubernetes heterogene.

1.2 Problema

Considerăm un **flux online** de execuții de workflow-uri într-un cluster Kubernetes heterogen. Deși numărul de **tipuri/template-uri de workflow** este **limitat** (workflows predefinite), fiecare tip se repetă de multe ori în timp, iar mai multe instanțe pot rula **concurrent** (real-time / near real-time).

Problema devine astfel dublă:

- **Optimizare per workflow:** minimizarea latenței end-to-end (makespan) pentru fiecare instanță, folosind contextul DAG (critical path) și evitarea penalităților de pornire.
- **Optimizare la nivel de sistem:** creșterea performanței globale când rulează multe

instanțe simultan (de exemplu reducerea latenței medii/ponderate și a tail latency, creșterea throughput-ului), sub contendență și politici de priorități între workflow-uri.

În plus, sistemul trebuie să fie **adaptiv**: pentru workflow-uri/task-uri care se repetă, scheduler-ul învață plasarea optimă și o reutilizează în execuțiile viitoare; pentru workflow-uri/task-uri noi, scheduler-ul explorează controlat pentru a învăța rapid unde sunt cele mai bune plasări.

Cum planificăm (când + unde) task-urile unui workflow pe noduri heterogene astfel încât să minimizăm latența end-to-end și latența de pornire, respectând compatibilitățile și contendența?

În această lucrare separăm explicit două noțiuni:

- **Localitatea datelor[17, 13] (data locality)**: unde se află datele de intrare/ieșire ale task-urilor (fișiere, obiecte, shard-uri) și costul de transfer pe muchiile DAG-ului.
- **Localitatea imaginilor[7] (image locality)**: dacă imaginea containerului este deja prezentă pe un nod, influențând latența de pornire (*cold start*).

În **baseline**, **ignorăm localitatea datelor** și presupunem un model de acces aproape uniform la date: există un strat de stocare partajat (de exemplu, un dispozitiv de rețea / object store / NFS) la care toate nodurile au acces similar, astfel încât costul de acces/transfer al datelor nu depinde semnificativ de plasare.

În schimb, **nu ignorăm localitatea imaginilor**: putem prefera noduri pe care imaginea este deja disponibilă, ca un semnal practic pentru reducerea time-to-running și a tail latency, fără a introduce complexitatea completă a modelării datelor de intrare/ieșire.

1.3 Motivație: de ce nu e suficient Argo + kube-scheduler default

Argo execută DAG-ul conform dependențelor, însă nu optimizează plasarea în cluster. Kube-scheduler optimizează placement-ul pod-urilor pe baza resurselor și a unor reguli declarative (affinity, taints/tolerations), dar nu are acces la:

- rolul task-ului în DAG (criticitate, poziție pe critical path);
- obiectivul end-to-end (minimizarea makespan-ului workflow-ului, nu doar plasare locală);
- clasele de startup-cost (cold/hot, image locality, prefetch).

Prin urmare, deciziile locale pot prelungi calea critică sau pot introduce cold-start-uri evitabile, crescând latența totală.

1.4 Obiective

Obiectivele lucrării sunt:

- Definirea unui model pentru execuția **online** a multor instanțe de workflow-uri (DAG) în Kubernetes heterogen, cu un set limitat de tipuri/template-uri care se repetă.
- Proiectarea unui mecanism **workflow-to-scheduler** pentru metadata (ID tip workflow, ID instanță, rank/criticitate, preferințe, startup-cost class, prioritate).
- Implementarea unei politici **workflow-aware placement** (critical-path / HEFT-inspired) pentru optimizare end-to-end.
- Implementarea unei bucle de **învățare online**:
 - *exploit*: reutilizarea profilului de plasare învățat pentru workflow-uri repetate;
 - *explore*: benchmark-on-miss pentru workflow-uri/task-uri noi și actualizarea preferințelor.
- Optimizare **la nivel de sistem** sub concurență: reducerea latenței medii/ponderate și a tail latency și creșterea throughput-ului, folosind admission control/queueing când e necesar.
- (Ulterior) Suport pentru **priorități per workflow** și **preemptie**, astfel încât workflows critice să poată avansa chiar sub concurență.

1.5 Întrebări de cercetare și ipoteze

- Poate un scheduler workflow-aware (cu semnale de critical path) să reducă makespan-ul față de plasarea locală pod-by-pod?
- În scenarii cu multe instanțe concurente, îmbunătățește această abordare **performanța globală** (latența medie/ponderată, throughput, p95/p99) comparativ cu baseline?
- Cât de mult contribuie cold-start aware placement (image locality, prefetch, warm cache) la reducerea time-to-running și tail latency când sistemul este încărcat?
- Învățarea online (explore/exploit) reduce costul deciziilor pe termen lung: workflow-urile repetate ajung să fie planificate aproape optim fără overhead de explorare?
- **ulterior**: Politicile de priorități și preemptie reduc degradarea pentru workflows critice fără a compromite sever fairness-ul pentru restul?

1.6 Soluția propusă

Soluția folosește Kubernetes ca platformă peste care adaugă un strat workflow-aware:

- **Metadata de workflow/task** (CRD sau annotations) pentru rank/criticitate, preferințe și startup-cost class.
- **Scheduler plugin** care scorează nodurile folosind semnale de heterogenitate + critical path + cold-start.
- **Mecanisme cold-start aware**: image locality, warm cache, prefetch (opțional lazy pulling / P2P).
- **Stage-level gang[10, 9, 5]** pentru task-urile ready din același stadiu (fork) când este benefic.

- **Admission control** la nivel de workflow (queueing/quotas).

1.7 Structura lucrării

Capitolul **State of the Art** prezintă planificarea în sisteme distribuite și elemente relevante pentru workflow-aware scheduling în Kubernetes. Capitolul **Arhitectură și Implementare** descrie componentele propuse: interfața workflow-to-scheduler[15, 14], plugin-ul de scheduling și mecanismele cold-start aware/gang/admission. Capitolul **Evaluare** definește setup-ul experimental și compară baseline-ul cu variantele propuse. Capitolul final discută concluzii, limitări și direcții ulterioare.

2 STATE OF THE ART

2.1 Planificare orientată pe workflow (DAG) pe resurse heterogene

O linie clasică de cercetare tratează planificarea workflow-urilor ca problemă de optimizare pe grafuri, unde obiectivul este minimizarea timpului total de execuție (makespan) sau a timpului de finalizare pe calea critică. Algoritmi precum HEFT[16] (*Heterogeneous Earliest Finish Time*) folosesc estimări de timp de execuție pe tipuri de resurse și prioritizează task-urile în funcție de un rank asociat DAG-ului, alegând plasarea care minimizează timpul de finalizare estimat. În context Kubernetes, provocarea este că scheduler-ul operează asupra pod-urilor individuale, iar informația de DAG trebuie propagată explicit.

2.2 Interfața workflow-to-scheduler: transmiterea contextului către scheduler

Pentru a permite unui scheduler să ia decizii workflow-aware, este necesar un contract prin care orchestratorul (sau un controller) expune metadata despre dependențe, criticitate și obiective. Conceptul de *Common Workflow Scheduler Interface (CWSI)* propune standardizarea acestor semnale astfel încât planificarea să poată fi separată de orchestrare, păstrând compatibilitatea cu mai multe sisteme[14]. Această lucrare adoptă o idee similară, implementată ca metadata în Kubernetes (CRD/annotations) consumate de un plugin din Scheduling Framework.

2.3 Co-scheduling (gang) pentru stadii fork-join

În workflows cu etape fork-join, pornirea parțială a unui set de task-uri ready poate produce utilizare inefficientă și creșterea latenței end-to-end, deoarece progresul real este limitat de straggler-e. Co-scheduling-ul (gang) tratează un grup de pod-uri ca o unitate de admitere/planificare: fie pornesc împreună, fie sunt amânate. În Kubernetes, astfel de mecanisme pot fi realizate prin extensii de tip Workload API / plugin-uri ce folosesc puncte de extensie precum *Permit* [9].

2.4 Admission control și queueing la nivel de workflow

În scenarii multi-tenant sau cu multiple workflows concurente, o decizie importantă este **care workflow** este admis în cluster, nu doar unde este plasat următorul pod. Mecanismele de queueing și quotas la nivel de workload (ex. Kueue) separă admiterea de plasarea efectivă, permițând politici de echitate și control al contendenței[4, 11]. Acest strat poate fi combinat cu scheduling-ul workflow-aware pentru predictibilitate și reducerea thrashing-ului.

2.5 Localitatea datelor vs. localitatea imaginilor

În literatura de planificare, **localitatea datelor** se referă la plasarea task-urilor cât mai aproape de datele de intrare/ieșire pentru a reduce costurile de transfer și a îmbunătăți performanța end-to-end. Exemple clasice includ mecanisme care întârzie temporar execuția pentru a obține localitate sau care optimizează global transferurile pe muchiile DAG-ului.

În Kubernetes, un concept separat este **localitatea imaginilor (image locality)**, care ține de disponibilitatea imaginii containerului pe nod și influențează în special latența de pornire (image pull). Deși ambele au numele de “localitate”, ele afectează sisteme diferite: data locality afectează costul de I/O și comunicare între task-uri, iar image locality afectează cold start. În această lucrare, modelăm explicit doar a doua categorie în baseline, păstrând data locality ca direcție de extindere.

2.6 Cold start și distribuția imaginilor: impact asupra latenței de pornire

Pentru task-uri scurte sau pipeline-uri sensibile la latență, overhead-ul de start (image pull, init, cold cache) devine dominant. Kubernetes poate prefera noduri cu imaginea deja disponibilă (*image locality*) și poate fi extins cu tehnici precum prefetch, *lazy pulling* (de exemplu, stargz snapshotter) sau distribuție P2P a imaginilor (de exemplu, Dragonfly) [12, 3]. Integrarea acestor semnale în scorarea nodurilor oferă o direcție practică de reducere a p95/p99 pentru time-to-running.

3 ARHITECTURĂ ȘI IMPLEMENTARE

3.1 Viziune generală

Sistemul propus adaugă context de workflow în decizia de placement din Kubernetes, fără a înlocui orchestrarea dependențelor realizată de Argo Workflows. Arhitectura este compusă din:

- **Workflow Controller / Hint Generator:** extrage structura DAG-ului, calculează semnale (rank/criticitate), atașează metadata pe task-uri/pod-uri.
- **Workflow-to-Scheduler Interface (CRD/Annotations):** contractul prin care semnalele ajung la scheduler.
- **Scheduler Plugin (Scheduling Framework):** integrează semnalele în filtrare/scorare pentru a decide plasarea.
- **Cold-start Manager:** menține warm cache, declanșează prefetch, monitorizează image locality.
- **Stage Co-scheduler:** aplică gang pentru seturi de task-uri ready când este benefic.
- **Admission Control:** mecanism de queueing/quotas la nivel de workflow.
- **Placement Policy Store (per workflow-template):** păstrează un profil învățat de plasare pentru fiecare tip de workflow și (opțional) pentru semnături de task (image, resurse, etapă). Profilul include ordini best/next-best și estimări de runtime/startup per tip de nod.
- **Online Learning & Controller:** decide când să *exploateze* profilul existent (pentru workflows repetate) și când să *exploreze* (benchmark-on-miss pentru task-uri noi), actualizând continuu profilurile.

3.2 Model: workflow, task și semnale

Un **workflow** este un DAG. Pentru fiecare task definim:

- **Hard constraints:** compatibilitate (GPU/CPU/IO), resurse (CPU/memorie/GPU), taints/tolerations.
- **Preferințe ordonate:** best/next-best tipuri de nod.
- **Semnale workflow-aware:** rank/criticitate (ex. pe critical path), deadline/priority (opțional), identificator workflow și stage.
- **Startup-cost class:** hot/cold, sensibilitate la image pull, toleranță la latență.

Notă despre date: un task poate avea date de intrare și date de ieșire, însă în această versiune nu modelăm explicit poziționarea acestor date în cluster. Presupunem un strat de stocare partajat cu acces similar din toate nodurile, astfel încât plasarea nu schimbă semnificativ costul de transfer al datelor pe muchiile DAG-ului. În consecință, scorarea nodurilor se bazează pe: (i) estimări de timp de execuție pe tipuri de nod (heterogenitate) și (ii) penalități de pornire (cold start).

Separat, considerăm **image locality** ca parte din cold-start aware scheduling: preferăm nodurile unde imaginea este deja prezentă sau poate fi adusă mai rapid (prefetch/lazy pulling), deoarece acest efect este observabil și important pentru latența de start.

3.3 Interfața workflow-to-scheduler (CRD/Annotations)

Implementăm metadata în Kubernetes sub forma:

- **Annotations** pe Pod/Template (ex.: workflow/id, task/id, task/rank, task/stage, startup/class);
- sau un **CRD** (ex.: WorkflowSchedulingHint) indexat după (workflow, task) pentru acces eficient.

3.4 Politica workflow-aware: critical-path / HEFT-inspired scoring

Plugin-ul din Kubernetes Scheduling Framework implementează o politică de scorare care combină:

- **Heterogenitate:** potrivirea tipului de nod cu preferințele task-ului;
- **Estimare de finish time:** preferință pentru noduri care minimizează *earliest finish time* estimat pentru task, ponderat cu rank-ul (criticitate);
- **Contendență:** penalizare pentru noduri congestionate sau cu resurse insuficiente, cu fallback next-best;
- **Stabilitate:** evitarea oscilațiilor prin smoothing (ex. EWMA pe estimări sau penalizări).

Această politică urmărește minimizarea makespan-ului workflow-ului, nu doar plasarea local optimă a unui pod.

3.5 Cold-start aware scheduling

Pentru a reduce time-to-running și tail latency, integrăm semnale și mecanisme precum:

- **Image locality:** preferăm noduri unde imaginea este deja disponibilă (când nu contrazice preferințele de heterogenitate).
- **Warm-instance caching (hot/cold):** menținerea unor instanțe sau resurse pregătite pentru task-uri frecvente.
- **Prefetch / pre-pull:** declanșarea proactivă a descărcării imaginilor pentru task-uri care devin eligibile curând.
- **Optional: lazy pulling / distribuție P2P:** reducerea timpului până la start prin tehnici de runtime/distribuție.

Aceste mecanisme sunt expuse scheduler-ului ca semnale (startup-cost class) și sunt integrate în scorare.

3.6 Stage-level co-scheduling (gang) pentru fork-join

În workflows cu etape fork-join, controller-ul poate grupa task-urile ready în stadii și poate cere admiterea/planificarea în mod coordonat:

- fie printr-o barieră logică (*Permit*) care pornește grupul împreună;
- fie printr-o regulă: dacă nu există suficiente resurse pentru un prag (k din n), amânăm stage-ul pentru a evita progres parțial.

Scopul este reducerea straggler-elor care prelungesc calea critică.

3.7 Learning loop: explore/exploit pentru workflows repetate

Sistemul tratează workflow-urile ca instanțe dintr-un set limitat de tipuri (template-uri) care se repetă. Pentru fiecare tip de workflow, menținem un **profil de plasare** (policy) care este actualizat incremental:

- **Exploit (repetiție):** pentru instanțe viitoare ale aceluiași workflow-template, scheduler-ul aplică direct ordinea de preferințe învățată (best/next-best) și estimările de cost (runtime + startup).
- **Explore (noutate / drift):** când apare un task nou (sau estimările devin incerte), rulăm controlat un set limitat de probe (benchmark-on-miss) pe tipuri de nod relevante, colectăm runtime și startup latency, apoi actualizăm profilul (EWMA / bandit simplu).

Prin această buclă, deciziile devin mai bune în timp, iar overhead-ul de explorare se amortizează pe repetitivitatea workload-ului.

3.8 Admission control

În scenarii cu multiple workflows concurente, includem un strat de admitere la nivel de workflow:

- workflow-ul intră într-o coadă; admiterea ține cont de quotas și priorități;
- numai workflows admise generează pod-uri executabile sau primesc permisiunea de planificare.

3.9 Optimizare globală sub concurență: priorități și (ulterior) preemptie

Deoarece în sistem rulează simultan multe instanțe de workflow, nu urmărim doar minimizarea makespan-ului unui workflow izolat, ci și îmbunătățirea performanței globale: latență medie/ponderată, throughput și tail latency.

Fiecare workflow poate avea o **prioritate** (sau greutate) care influențează:

- ordinea de admitere (admission) în sistem când resursele sunt limitate;
- preferința pentru resurse rapide pentru task-uri critice (pe critical path);
- (ulterior) decizii de **preemptie** pentru a elibera resurse pentru workflows cu prioritate ridicată.

În versiunea inițială, tratăm prioritățile prin admitere și scorare (fără preemptie). Ca extensie, se poate integra preemptie astfel încât workflows critice să poată progresa sub încărcare, cu reguli care limitează impactul asupra fairness-ului și stabilității sistemului.

4 EVALUARE EXPERIMENTALĂ

4.1 Setup experimental

Evaluarea include scenarii în care instanțe de workflow-uri sosesc online și rulează concurrent:

- **Repetitivitate:** un set limitat de workflow-template-uri este repetat de multe ori; măsurăm cum se îmbunătățește performanța în timp prin exploit.
- **Noutate:** introducem periodic workflow-uri sau task-uri noi (image nouă / profil nou) și măsurăm costul de explore și viteza de convergență către plasări bune.
- **Priorități:** mix de workflows cu priorități diferite; evaluăm latența ponderată și (ulterior) impactul preempției.

Pentru izolarea efectelor, **nu introducem constrângeri de data locality**: datele de intrare/ieșire sunt accesate dintr-un strat de stocare partajat, cu acces comparabil din toate nodurile. Astfel, diferențele observate sunt atribuite în principal heterogenității, contendenței și penalităților de pornire. În schimb, **image locality** este variată controlat (imagini deja prezente pe un subset de noduri vs. cold pull), pentru a cuantifica impactul asupra time-to-running și tail latency.

4.2 Variante comparate

- **Baseline:** Argo + kube-scheduler default + reguli declarative minime (constraints/preferințe statice).
- **Workflow-aware:** plugin de scorare cu semnale critical-path/rank + preferințe best/next-best.
- **Cold-start aware:** workflow-aware + image locality + prefetch/warm cache (și opțional lazy pulling/P2P).
- **Gang/stage:** workflow-aware + co-scheduling pe stadii fork.
- **Extins:** admission control la nivel de workflow (sub contendență).

4.3 Metrici

- **End-to-end:** makespan per workflow, latență pe calea critică.
- **Startup:** time-to-scheduled, time-to-running, cold-start rate.
- **Tail:** p95/p99 pentru start și pentru makespan.

- **Resurse:** utilizare CPU/memorie, rate de pending, preemption/retries.
- **Stabilitate:** variabilitate între rulări, thrashing sub contendență.
- **Global:** latență medie și latență **ponderată** pe priorități, throughput (workflows/min), time-in-queue.
- **Learning:** regret/îmbunătățire în timp (diferența față de policy stabilizată), costul explorării, timpul de convergență.

4.4 Ipoteze testate

Se validează ipotezele 1–5 prin experimente controlate:

- efectul semnalelor de DAG (rank) asupra makespan-ului;
- efectul semnalelor de cold-start asupra time-to-running și tail latency;
- beneficiul gang/stage în fork-join (reducerea straggler-elor);
- efectul admission control în scenarii multi-workflow.

4.5 Amenințări la validitate

- estimările de runtime pot fi eronate (se atenuează prin learning loop);
- workload-urile sintetice pot să nu acopere toate cazurile reale;
- rezultatele depind de configurația clusterului (se raportează explicit).

5 LIMITĂRI ȘI DIRECȚII ULTERIOARE

Deși lucrarea urmărește să introducă context de workflow în placement și să reducă cold start, există limitări și extensii naturale:

- **Localitatea datelor:** modelarea costului de transfer pe muchiile DAG-ului și preferințe de co-locare (zonă/rack) când comunicarea domină.
- **Interferență avansată:** integrarea unor modele mai precise pentru co-locare (noisy neighbor), inclusiv semnale din cgroup, IO scheduler și observabilitate detaliată.
- **Predicție și învățare:** modele de predicție runtime (features + regression) și politici mai sofisticate, inclusiv per-workflow adaptation.
- **Obiective multi-criteriu:** optimizare simultană pentru latență, cost, energie, fairness și SLO-uri.
- **Multi-cluster / edge:** extinderea către scheduling între clustere sau edge-cloud, cu constrângeri de latență de rețea.

BIBLIOGRAFIE

- [1] Argo workflows documentation: Dag. <https://argo-workflows.readthedocs.io/en/latest/walk-through/dag/>. Accessed 2026-02-12.
- [2] Argo workflows: The workflow engine for kubernetes. <https://argoproj.github.io/workflows/>. Accessed 2026-02-12.
- [3] Dragonfly: P2p-based image and file distribution (cncf). <https://github.com/dragonflyoss/Dragonfly>. Accessed 2026-02-12.
- [4] Introducing kueue: Job queueing controller. <https://kubernetes.io/blog/2022/10/04/introducing-kueue/>. Accessed 2026-02-12.
- [5] Kubernetes documentation: Gang scheduling. <https://kubernetes.io/docs/concepts/scheduling-eviction/gang-scheduling/>. Accessed 2026-02-12.
- [6] Kubernetes documentation: kube-scheduler. <https://kubernetes.io/docs/concepts/scheduling-eviction/kube-scheduler/>. Accessed 2026-02-12.
- [7] Kubernetes documentation: Scheduler configuration (plugins incl. imagelocality). <https://kubernetes.io/docs/reference/scheduling/config/>. Accessed 2026-02-12.
- [8] Kubernetes documentation: Scheduling framework. <https://kubernetes.io/docs/concepts/scheduling-eviction/scheduling-framework/>. Accessed 2026-02-12.
- [9] Kubernetes documentation: Workload api (v1.35 alpha). <https://kubernetes.io/docs/concepts/workloads/workload-api/>. Accessed 2026-02-12.
- [10] Kubernetes v1.35: Introducing workload aware scheduling. <https://kubernetes.io/blog/2025/12/29/kubernetes-v1-35-introducing-workload-aware-scheduling/>. Accessed 2026-02-12.
- [11] Kueue documentation: Concepts. <https://kueue.sigs.k8s.io/docs/concepts/>. Accessed 2026-02-12.
- [12] stargz-snapshotter (containerd): Lazy pulling of container images. <https://github.com/containerd/stargz-snapshotter>. Accessed 2026-02-12.
- [13] Michael Isard, Vijayan Prabhakaran, Jon Currey, Udi Wieder, Kunal Talwar, and Andrew Goldberg. Quincy: Fair scheduling for distributed computing clusters. In *Proceedings of*

the ACM SIGOPS 22nd Symposium on Operating Systems Principles (SOSP '09). ACM, 2009.

- [14] Fabian Lehmann, Jonathan Bader, Lauritz Thamsen, and Ulf Leser. The common workflow scheduler interface: Status quo and future plans. In *Proceedings of the SC '23 Workshops (SC-W '23)*, 2023. Accessed 2026-02-12.
- [15] Fabian Lehmann, Jonathan Bader, Friedrich Tschirpke, Lauritz Thamsen, and Ulf Leser. How workflow engines should talk to resource managers: A proposal for a common workflow scheduling interface. In *2023 IEEE/ACM 23rd International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*, 2023. Accessed 2026-02-12.
- [16] Haluk Topcuoglu, Salim Hariri, and Min-You Wu. Performance-effective and low-complexity task scheduling for heterogeneous computing. *IEEE Transactions on Parallel and Distributed Systems*, 13(3):260–274, 2002.
- [17] Matei Zaharia, Dhruba Borthakur, Joydeep Sen Sarma, Khaled Elmeleegy, Scott Shenker, and Ion Stoica. Delay scheduling: A simple technique for achieving locality and fairness in cluster scheduling. In *Proceedings of the 5th European Conference on Computer Systems (EuroSys '10)*. ACM, 2010. Accessed 2026-02-12.

...